

DSAIL- HW9

Tomas Salaz

April 6, 2026

Problem 1

Suppose the Greedy Algorithm outputs a correct Matrix called A . Suppose there is another Matrix B that is an optimal solution.

Let row i be the first row where A and B differ. Let j and j' be columns in row i such that $A[i, j] = B[i, j'] = 1$ and $A[i, j'] = B[i, j] = 0$.

Since the Greedy Algo, chooses column j over j' we know $c_j - a_j \geq c_{j'} - a_{j'}$, this implies that column j has at least as much remaining demand as column j' .

where a_j and $a_{j'}$ are the number of 1's already placed in Cols j and j' in rows $1..i - 1$. So Col j has, at minimum, as much remaining demand as Col j'

Because both A and B have the same Col Sums, Col j must have exactly $c_j - a_j$ 1's in rows $i..n$ in both matrices. Col j' must have exactly $c_{j'} - a_{j'}$ 1's in the same rows.

Since $B[i, j] = 0$ and $A[i, j] = 1$, Matrix B must have replace the missing 1 from the Col in another Row, i' where $i' > i$.

Thus there exists Row i' such that $B[i', j] = 1$ and $B[i', j'] = 0$ so the Col and Row Sums remain satisfied.

Now we can take the 4 coords, $(i, j), (i, j'), (i', j), (i', j')$ and exchange them in Matrix B , such that:

$B[i, j] \leftarrow 1, \quad B[i, j'] \leftarrow 0, \quad B[i', j] \leftarrow 0, \quad B[i', j'] \leftarrow 1$. (OR we can swap $1 \rightarrow 0$ and $0 \rightarrow 1$)

These exchanges preserves the Sums, Thus still a valid solution.

This Exchange makes row i in Matrix B the same as i in Matrix A while keeping the sums in tact. By repeatedly applying these exchange arguments, we can transform B into A without violating the sums constraints. Hence, we keep the correct sums and we have proved that the Greedy Solution is Optimal as long as there is a valid configuration.

Problem 2

Using the Tax Method

We can assign each operation, insertion/deletion, a fixed Amortized Cost, C , where the cost is $O(1)$, and the surplus is put into the bank to 'pay' for the

table rebuild.

After a Doubling, the old table has size $m/2$ and the rebuild trigger since the elements, k , became larger than $3/4$. So $k \geq 3/4 \times m/2 = 3m/8$, so the load factor, $T \approx 3/8$

After a Halving, the old table has size $2m$ and the rebuild was triggered because k became smaller than $1/4$. So $k \leq 1/4 \times 2m = m/2$, so the load factor, $T \approx 1/2$

The minimum operations between rebuilds come from the starting table of size m with load factor between $3/8$ and $1/2$.

For a Doubling we need $3m/4$ elements, starting from at most $m/2$ elements, so $3m/4 - m/2 = m/4$, so we need at least $m/4$ operations.

For a Halving we need $m/4$ elements, starting from at least $3m/8$ elements, so $3m/8 - m/4 = m/8$, so we need at least $m/8$ operations.

The cost of a rebuild is the size $O(m)$ and the cost of Reinsertion all elements cost $O(1)$ per element with at least $3m/4$ elements giving, $O(m)$ total cost.

Between any 2 resizing operations, there are at least $O(m)$ insertions or deletions. Each resizing operations requires the same $\Omega(m)$ time to rebuild the table.

We assign a constant Amortized cost to each operation, $O(1)$ for the operation itself as well as the extra tax. Over $\Omega(m)$ operations, this tax accumulates to $O(m)$ total which is enough to pay for the resizing. Thus the amortized cost per operation is $O(1)$.

Problem 3

Both using Aggregate Method **Part A**

$f(n) = n$ if n is a power of 2, $f(n) = 1$ otherwise

The total cost of the first n operations is:

$$T(n) = \sum_{k=1}^n f(k) = \sum_{k=1}^n 1 + \sum_{k=1}^n k$$

The 1st sum is when k is not a power of 2, and the 2nd is if k is a power of 2.

In the 1st sum there is at most n cheap operations. For the second sum, the powers of 2 that are at most n are $1, 2, 4, \dots, 2^{\lfloor \log_2 n \rfloor}$ whose sum is the geometric series

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = 2^{\lfloor \log_2 n \rfloor + 1} - 1 \leq 2n - 1$$

Thus, $T(n) \leq n + (2n - 1) = O(n)$

So the amortized cost per operation is $T(n)/n = O(1)$

Part B

$f(n) = n^2$ if n is a power of 2, $f(n) = 1$ otherwise

The total cost of the first n operations

$$T(n) = \sum_{k=1}^n f(k) = \sum_{k=1}^n 1 + \sum_{k=1}^n k^2$$

In the same manner, the 1st sum is when k is not a power of 2 and the 2nd sum is when k is.

The 1st sum is at most n while the 2nd sum are powers of 2 at most, $1, 2, 4, \dots, 2^{\lfloor \log_2 n \rfloor}$

so we compute the sum again

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} (2^i)^2 = \sum_{i=0}^{\lfloor \log_2 n \rfloor} 4^i$$

This is as geometric series with ratio 4 so we can reduce the sum

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 4^i = \frac{4^{\lfloor \log_2 n \rfloor + 1} - 1}{4 - 1} = \frac{4 \cdot 4^{\lfloor \log_2 n \rfloor} - 1}{3}$$

Since $2^{\log_2 n} \leq n$ we have that $4^{\lfloor \log_2 n \rfloor} = (2^{\lfloor \log_2 n \rfloor})^2 \leq n^2$, Then we can reduce the sum further

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 4^i \leq \frac{4n^2 - 1}{3} \leq \frac{4n^2}{3}$$

Combining the parts, we get

$$T(n) \leq n + 4n^2/3 = O(n^2)$$

So the Amortized cost per operation is

$$T(n)/n = O(n)$$

Problem 4

A Data Structure for to support the 3 operations can be implemented using a modified Union Find. We maintain an initial array, initialized with 0's, $A[1..n]$ Then we use a Union Find Structure over the elements $[1, 2..n + 1]$ where index $n + 1$ is an invariant, set to 0. This structure has the same operations as the Union Find in the class slides (Union, Find, MakeSet).

Initially each index i is its own single set with representative i . For any index i the representative of the set containing i is the smallest index $j \geq i$ such that $A[j] = 0$. If the index doesn't exist, then the representative for that index is index $n + 1$ (invariant).

This property holds initially since every index is initially 0.

GetValue

This operation is constant time since we use an array store the values.

SetToOne

There are 2 parts of this operation

1. This operation takes constant time, $O(1)$ since we just access an element and write the value to 1.

2. We call **UNION**(i , $i+1$) to merge i into the set of $i + 1$.

When we call **UNION**, we make the index i 's parent pointer point to $i + 1$. This is done to 'link' the newly occupied position to its right neighbor so that we can find the Closest Right 0. This operation is done in the UnionFind Structure.

GetClosestRightZero

We call **FindSet**(i) and set that to x .

Let $x = \text{FindSet}(i)$. If $x = n + 1$, then no such index exists. Otherwise, return x .

This operation is done in the UnionFind structure as well.

Analysis: Correctness (GetClosestZero)

BC: Initially $A[i] = 0$ for all i and each element is its own representative via **MakeSet**. When we call **FindSet** each return is its own i (itself) thus this is trivially True.

IH: Induction on k , number of **SetToOne** operation preformed thus far

After k calls to **SetToOne**, the data structure holds every index i from $[1..n]$, **FindSet** returns the Closest Right 0

IS:

Before the call to UNION, FindSet returns some $j \geq i$ with $A[j] = 0$, by IH. Since $A[i] = 1$, the next closest 0 is at or after j .

After the call to UNION, FindSet(i) = FindSet(i+1)=j which is correct.

For any k not in i 's set, the union doesn't effect the pointers, so the property still holds, thus Correct.

Analysis: Run Time

GetTime runs in $O(1)$ time, is an array lookup

SetToOne runs in $O(1)$ in the worst case since it is just an array write. The UnionFind Part of this operation is included in the amortized analysis below.

GetClosestRightZero runs in $O(\log^* n)$ time in the worst case.

The runtime for MakeSet operation is $n + 1$ for initialization.

At most, n SetToOne operations each perform one UNION call, specifically the Path Compression union from the lecture slides and m GetClosestRightZero operation using the Path Compression FindSet.

Using the Union-Find with Path Compression, a sequence of m operations on n elements takes $O((m + n) \log^* n)$ time. Each SetToOne operations performs at most one UNION, and each GetClosestRightZero performs Find once.

Thus:

SetToOne runs in $O(1)$ amortized time

GetValue runs in $O(1)$ time in the worst case

GetClosestRightZero runs on $O(\log^* n)$ amortized time